Naive Bayes Classification of Mushrooms

Teoman Kaman

April 2025

Abstract

In this project, I implemented a Naive Bayes classifier from scratch to predict whether a mushroom is edible or poisonous using the UCI Mushroom dataset. I evaluated the model under two conditions: using all available features and using a reduced subset of 10 features. I did not use any machine learning libraries during training or prediction. The results showed high accuracy in both settings, demonstrating that Naive Bayes is effective for categorical classification.

1 Introduction

Naive Bayes is a probabilistic classification method based on Bayes' Theorem. It assumes that features are conditionally independent given the class label. Despite its simplicity, it performs well on many real-world datasets, especially when the features are categorical. In this project, I used the UCI Mushroom dataset, which includes attributes describing mushrooms, to classify them as edible or poisonous.

2 Methodology

First, I loaded the dataset from the UCI Machine Learning Repository and gave each column a name. There are 23 columns in total, including one for the class label and 22 features. I noticed that some entries in the 'stalk-root' column had missing values represented by a "?", so I removed those rows to clean the data.

After cleaning, I encoded all categorical values into integers. This step was important because Naive Bayes works with numerical values. I created a dictionary for each column that mapped every unique category to a number. For example, if "smooth" and "scaly" are values in the cap surface column, they might be encoded as 0 and 1.

Next, I split the dataset into features (X) and labels (y), and divided these into training and test sets. I used 60% of the data for training and the remaining 40% for testing.

2.1 Training Phase

In this phase, I computed the probabilities that the model would use to make predictions:

- The prior probability $P(c)$, which is the chance of a mushroom being edible or poisonous based on how often each class appears in the training data.
- The conditional probabilities $P(x_j|c)$, which is the probability of seeing a specific value of a feature given the class. For example, the probability of a mushroom having a white spore print given that it is edible.

To avoid errors when a value never appeared in the training set, I used Laplace smoothing, which adds a count of one to each feature value. This ensures all probabilities are defined.

2.2 Prediction Phase

For each test sample, I used the Naive Bayes formula:

$$\log P(c) + \sum_j \log P(x_j|c)$$

I computed the log probability for both classes (edible and poisonous) and chose the one with the higher score. This method allowed me to make predictions based on the values of each feature in a test sample.

3 Experiments and Results

3.1 Task 1: Full Feature Set

In the first experiment, I used all 22 features in the dataset. After training the model on 60% of the data and testing on the remaining 40%, I compared the predicted labels with the actual labels.

The model was able to correctly predict most of the test samples. The calculated accuracy was:

- **Accuracy:** 93.00%

This means the model predicted the correct class (edible or poisonous) in 93 out of every 100 mushrooms in the test set. While not perfect, this shows that the Naive Bayes classifier worked quite well even when using all the features.

3.2 Task 2: Reduced Feature Set

For the second task, I selected only 10 features that seemed more relevant to the classification. These features were: `cap-shape`, `cap-surface`, `cap-color`, `odor`, `gill-size`, `gill-color`, `stalk-root`, `veil-color`, `spore-print-color`, and `class`.

I repeated the same steps as before: I encoded these features, split the data into training and testing parts, trained the model, and used it to make predictions. I used a separate prediction function that works the same way but uses only the reduced number of features.

The accuracy was even higher in this experiment:

- **Accuracy:** 94.77%

This shows that removing some features can actually improve the model. In this case, the features I removed may have added noise or confusion. The remaining 10 features were more informative, especially features like `odor` and `spore-print-color`, which seem to be strongly linked to whether a mushroom is edible or poisonous.

Setting	**Number of Features**	**Accuracy**
Full Dataset	22	93.00%
Reduced Dataset	10	94.77%

Table 1: Accuracy comparison of Naive Bayes classifier

4 Discussion

The classifier performed slightly better using only 10 selected features. I believe this is because some of the full dataset features are redundant or less informative. Features like `odor` and `spore-print-color` had strong predictive power. By using a smaller but more meaningful set of features, the classifier became more accurate and computationally efficient.